

CSE494

Intelligent NoSQL Databases

UNIT II — MASTERCLASS

MongoDB Basics and CRUD Operations

Prepared as personal course-preparation material
Session 2026–27

Contents

- 1. MongoDB Core Structure: Databases, Collections, Documents
- 2. BSON Data Types — JSON vs. BSON Deep Dive
- 3. Database & Collection Lifecycle
- 4. CRUD Operations — Create
- 5. CRUD Operations — Read (Queries, Operators, Cursors)
- 6. CRUD Operations — Update
- 7. CRUD Operations — Delete
- 8. Comic-Style Scenario: The BSON Postal Service
- 9. Data Modeling, Indexes & Query Optimization Basics
- 10. Big Unified Example: University Course Portal (End-to-End)
- 11. Practical Exercises (with Solutions)
- 12. Self-Assessment: Conceptual Review Questions (with Model Answers)
- 13. Industry Interview & Viva Question Bank
- 14. Frequently Confused Concepts — Quick Clarifications
- 15. Full End-to-End Practical Walkthroughs (Terminal → CRUD → Indexing)

1. MongoDB Core Structure: Databases, Collections, Documents

1.1 The Three-Level Hierarchy

Every piece of data you ever store in MongoDB lives inside a strict three-level container hierarchy. Understanding this hierarchy cold is the single most important prerequisite for everything else in this unit — every command you type either creates, reads, updates, or deletes something at one of these three levels.

- Database — the top-level container. A single MongoDB server (mongod) can host many databases, each completely isolated from the others (separate namespace, separate files on disk).
- Collection — a named group of documents inside a database, roughly analogous to a 'table,' but without a fixed schema. A database can contain many collections.
- Document — a single BSON record inside a collection, roughly analogous to a 'row.' Each document is a self-contained set of field-value pairs, and different documents in the SAME collection are allowed to have completely different fields.

```
MongoDB Server (mongod)
|
+-- Database: "college"
|   |
|   +-- Collection: "students"
|       |
|       +-- Document { _id: 1, name: "Priya", branch: "CSE" }
|       +-- Document { _id: 2, name: "Arjun", branch: "ECE", cgpa: 8.7 }
|   |
|   +-- Collection: "courses"
|       +-- Document { _id: 1, code: "CSE494", credits: 4 }
|
+-- Database: "admin"
+-- Database: "local"
```

1.2 Mapping MongoDB Concepts to SQL — A Precise Translation Table

If you already know relational databases, the fastest way to onboard into MongoDB is a direct concept-to-concept mapping. Use this table constantly during the first few weeks — but also read the caveats below it, because the mapping is a teaching aid, not a perfect equivalence.

SQL Concept	MongoDB Equivalent	Key Difference
Database	Database	Conceptually identical — an isolated namespace of related data.
Table	Collection	A collection has NO fixed schema; documents inside it can vary in structure.
Row	Document	A document is a rich, nested BSON object (can contain arrays and sub-documents), not just a flat list of scalar values.
Column	Field	Fields are NOT declared in advance; each document independently defines which fields it has.

SQL Concept	MongoDB Equivalent	Key Difference
Primary Key	<code>_id</code> field	Every document automatically gets a unique <code>_id</code> (an ObjectId by default) acting as its primary key.
Foreign Key + JOIN	Embedding OR Manual Reference + <code>\$lookup</code>	MongoDB has no enforced foreign keys; relationships are either embedded directly or manually referenced and joined via the aggregation framework's <code>\$lookup</code> stage.
Schema (CREATE TABLE)	Optional JSON Schema Validation	Schema enforcement in MongoDB is opt-in per collection, not mandatory before any data can be inserted.

PRODUCTION NOTE: Caveat: don't over-apply this mapping. A 'collection' is not a table with unenforced columns — it is a fundamentally different design unit meant to hold self-contained, query-optimized objects. Good MongoDB schema design (Unit III) often looks NOTHING like a naive one-collection-per-SQL-table port.

1.3 Anatomy of a Single Document

Let's dissect one real document field by field, since every CRUD operation in this unit ultimately manipulates structures exactly like this one.

```
{
  "_id": ObjectId("64f1a2b3c4d5e6f7a8b9c0d1"), // unique identifier (primary key)
  "name": "Priya Sharma",                      // simple string field
  "age": 21,                                    // simple numeric field
  "isActive": true,                            // boolean field
  "enrolledOn": ISODate("2026-01-15T00:00:00Z"), // native Date type
  "address": {                                 // embedded (nested) sub-document
    "city": "Ludhiana",
    "state": "Punjab"
  },
  "courses": ["CSE494", "CSE401"],             // array of scalar values
  "grades": [                                  // array of embedded documents
    { "course": "CSE494", "grade": "A" },
    { "course": "CSE401", "grade": "B+" }
  ]
}
```

- Field names are always strings, written in double quotes in strict JSON representation (mongosh lets you omit quotes for simple identifier-style names).
- Values can be any BSON type — scalars (string, number, boolean, date), arrays, or entire nested sub-documents — and can nest to arbitrary depth (though MongoDB recommends staying under 100 levels and, practically, far shallower for performance).
- Maximum document size is 16MB — a hard BSON limit designed to prevent any single document from monopolizing memory/bandwidth. (For larger binary content, GridFS is used — outside this unit's scope.)
- Maximum nesting depth is 100 levels — rarely a practical constraint, but worth knowing as a hard system limit.

1.4 Going Deeper: Why Collections Have No Fixed Schema (and What That Really Buys You)

It's worth pausing on WHY MongoDB's designers chose schema flexibility at the collection level, rather than treating it as an accident of implementation. In a relational table, adding a new column to accommodate a new feature requires an ALTER TABLE statement — an operation that, on a large production table, can lock the table or take minutes to hours to complete, causing real application downtime.

In MongoDB, adding a new field to your application's documents requires ZERO schema migration — you simply start writing documents that include the new field. Older documents that don't have it are completely unaffected and remain valid; your application code handles their absence with a default value (e.g., `doc.newField || defaultValue`). This is what "schema-on-read" (introduced in Unit I) looks like in practice, and it is the single biggest reason fast-iterating product teams choose MongoDB during a product's early, rapidly-changing phase.

NOTE: This flexibility is a double-edged sword: it removes migration friction but shifts the responsibility for data consistency onto the APPLICATION layer. A disciplined team still maintains an informal or validator-enforced schema in practice — 'no fixed schema' does not mean 'no schema at all,' it means 'the database won't stop you from being inconsistent, so your code and validation rules must.'

2. BSON Data Types — JSON vs. BSON Deep Dive

2.1 What Is BSON, Really?

BSON stands for Binary JSON. It is the binary-encoded serialization format MongoDB actually uses to store documents on disk and transmit them over the wire — every document you see printed as pretty JSON in mongosh is, underneath, stored as BSON. BSON was designed by MongoDB's creators specifically to solve three problems plain-text JSON has as a database storage format.

2.2 Why BSON Instead of Plain JSON?

Problem with plain-text JSON	How BSON solves it
JSON has only one numeric type (a generic 'number'), which is ambiguous and lossy for large integers or precise decimals.	BSON has explicit numeric types: Int32, Int64, Double, and Decimal128 (for exact financial-grade decimal precision), so numeric intent is never lost or misinterpreted.
JSON has no native date type — dates must be encoded as strings and re-parsed on every read, which is slow and error-prone across time zones.	BSON has a native Date type, stored as a 64-bit integer (milliseconds since the Unix epoch), enabling fast, unambiguous date arithmetic and range queries.
Parsing text-based JSON (scanning character by character, detecting delimiters) is CPU-expensive at scale, especially for a database serving millions of ops/sec.	BSON is binary and length-prefixed — the parser can jump directly to a field or skip over an entire embedded sub-document without scanning every character, making traversal dramatically faster.
Plain JSON cannot represent binary data (images, files) without inefficient Base64 text-encoding, which inflates size by ~33%.	BSON has a native BinData type to store raw binary bytes directly, with no text-encoding overhead.
JSON has no unique-identifier type, forcing applications to invent their own ID conventions.	BSON has a dedicated ObjectId type — a compact, globally-unique 12-byte identifier generated efficiently without a central coordinator.

NOTE: Trade-off to remember for exams: BSON documents are typically **SLIGHTLY LARGER** on disk than the equivalent minified JSON text (due to storing field-length prefixes and type tags), but they are dramatically **FASTER** to parse, traverse, and query — a deliberate, favorable trade for a database engine, where read/traversal speed matters far more than a few extra bytes.

2.3 The Complete BSON Type Reference

Below are the BSON types most relevant to this course, with their mongosh constructor syntax where applicable.

BSON Type	Description	Example (mongosh syntax)
String	UTF-8 text data — the most common field type.	"name": "Priya Sharma"
Int32 / Int64 ("NumberInt"/"NumberLong")	32-bit or 64-bit signed integers.	"age": 21 // stored as Int32 automatically
Double	64-bit floating point — the default numeric type for decimals typed directly in mongosh.	"cgpa": 8.75

The default `_id` value MongoDB auto-generates is far from a random string — it is a carefully engineered 12-byte value designed to be globally unique **WITHOUT** requiring a central coordinating server (important for a distributed database where many nodes might insert documents concurrently).

```

ObjectId("64 f1 a2 b3 c4 d5 e6 f7 a8 b9 c0 d1")
      |_____| |_____| |_____|
      |         |         |
      4-byte Unix   5-byte random   3-byte counter
      timestamp     machine/process (incrementing
      (seconds since identifier per-process,
      epoch) starts random)

Total: 12 bytes = 24 hex characters when printed

```

- Page 7 of 51

- Because generation doesn't require a round-trip to the server, drivers can generate the `_id` CLIENT-SIDE before even sending the insert — useful for referencing a document's ID immediately after constructing it in application code.

NOTE: Interview-relevant detail: you can extract the embedded creation timestamp from any `ObjectId` using `.getTimestamp()` in mongosh — e.g., `ObjectId("...").getTimestamp()` — without needing a separate `createdAt` field, though explicitly storing `createdAt` is still standard best practice for clarity and query-ability.

2.5 Checking a Field's Type in mongosh

```
// The $type operator lets you query documents by the BSON type of a field
db.students.find({ age: { $type: "int" } })

// You can also check type using the JS-level typeof from an inspected document
let s = db.students.findOne()
typeof s._id      // 'object' (ObjectId is a JS object wrapper in mongosh)
typeof s.name     // 'string'
```

2.6 Going Deeper: A Common Type Pitfall — Numbers Stored as Strings

A very frequent real-world bug is inserting a numeric-looking value as a `STRING` by accident (e.g., from an HTML form or a CSV import where every field arrives as text), then being confused why numeric range queries like `{ age: { $gt: 21 } }` silently return zero results.

```
// Accidentally inserted 'age' as a STRING instead of a number
db.students.insertOne({ name: "Bug Example", age: "22" }) // "22" is a STRING, not Int32

// This query returns NOTHING for the document above,
// because MongoDB compares by BSON TYPE first -- a string is never > a number
db.students.find({ age: { $gt: 21 } })

// Fix at the application layer -- convert before inserting
db.students.insertOne({ name: "Fixed Example", age: parseInt("22", 10) })
```

PRODUCTION NOTE: This is one of the most common production bugs teams encounter with schema-flexible databases, and it is exactly the class of problem `$jsonSchema` validation (Section 3.3) is designed to catch at write time — requiring `bsonType: "int"` on the age field would reject the string "22" outright rather than silently storing bad data that breaks queries later.

3. Database & Collection Lifecycle

3.1 Creating and Switching Databases: use <db>

Unlike SQL's explicit CREATE DATABASE statement, MongoDB has no dedicated 'create database' command. Instead, a database is created IMPLICITLY the moment you first store data in it.

```
// Switch to (or 'create') a database named 'college'
use college

// At this point, 'college' does NOT yet physically exist on disk!
// MongoDB only materializes it once you write actual data.
show dbs    // 'college' will NOT appear here yet

// Writing data finally creates the database (and collection) for real
db.students.insertOne({ name: "Priya", branch: "CSE" })

show dbs    // NOW 'college' appears in the list
```

PRODUCTION NOTE: Classic beginner trap: running `use mydb` and then immediately `show dbs` and being confused that 'mydb' isn't listed. This is expected behavior — MongoDB is lazy about creating databases; nothing is written to disk until the first document (or explicit collection) is created.

3.2 Implicit Collection Creation

Just like databases, collections are usually created IMPLICITLY — simply calling an insert method on a collection name that doesn't exist yet causes MongoDB to create it automatically, with sensible default storage settings.

```
use college

// 'assignments' collection doesn't exist yet – no error is thrown
db.assignments.insertOne({ title: "Unit II Lab", dueDate: ISODate("2026-09-01") })

// It now exists, created implicitly with default options
show collections    // 'assignments' now appears
```

3.3 Explicit Collection Creation: db.createCollection()

While implicit creation is convenient for quick prototyping, PRODUCTION schemas usually create collections explicitly using `db.createCollection()`, because this is the only way to attach configuration options at creation time — most importantly, schema validation rules, capped-collection settings, and collation.

```
// Basic explicit creation (functionally similar to implicit creation)
db.createCollection("students")
```

```
// Explicit creation WITH schema validation rules attached
db.createCollection("students", {
  validator: {
    $jsonSchema: {
      bsonType: "object",
      required: ["name", "email"],
      properties: {
        name: { bsonType: "string", description: "must be a string and is required"
      },
      email: { bsonType: "string", pattern: "^.+@.+$", description: "must be a
valid email and is required" },
      age: { bsonType: "int", minimum: 0, maximum: 120, description: "must be an
integer between 0-120" }
    }
  },
  validationLevel: "strict",    // reject any non-conforming insert/update
  validationAction: "error"    // reject the write outright (vs. "warn", which
only logs)
})

// Explicit creation of a CAPPED collection (fixed-size, auto-evicts oldest documents
// -- e.g., MongoDB's own oplog.rs uses exactly this mechanism internally)
db.createCollection("activityLog", { capped: true, size: 5242880, max: 1000 })
```

NOTE: validationLevel: "strict" applies validation to ALL inserts and updates; "moderate" applies it only to inserts and to updates of documents that ALREADY satisfied the schema (useful when migrating a collection with pre-existing, imperfect legacy data). validationAction: "warn" logs a warning but still allows the write — useful for testing a new rule before enforcing it.

3.4 Dropping Collections: db.<collection>.drop()

```
// Permanently delete a collection AND all documents/indexes inside it
db.assignments.drop()
// returns true on success, false if the collection didn't exist
```

PRODUCTION NOTE: drop() is IRREVERSIBLE and deletes all indexes along with the data — there is no built-in 'trash' or undo. Always confirm you're connected to the correct database (check with `db`) before running any drop command, and prefer taking a backup (mongodump) before destructive operations in any environment that matters.

3.5 Dropping an Entire Database: db.dropDatabase()

```
// Make sure you're pointed at the RIGHT database first!
db
// college

// Permanently delete the ENTIRE current database — all collections, all documents,
all indexes
db.dropDatabase()
```

```
// { ok: 1, dropped: 'college' }
```

PRODUCTION NOTE: This is one of the single most dangerous commands in all of MongoDB. There is no confirmation prompt. A very common real-world horror story is running `db.dropDatabase()` while accidentally connected to a PRODUCTION cluster instead of a local/staging one. Production teams typically restrict this command via role-based access control (RBAC) so only specific admin roles can execute it.

3.6 Listing & Inspecting Databases and Collections

```
// List all databases on the server, with sizes
show dbs

// List all collections in the CURRENT database
show collections

// Get detailed stats about the current database (storage size, collection count,
etc.)
db.stats()

// Get detailed stats about one specific collection
db.students.stats()

// Check whether the current database actually exists on disk yet
db.getName()
```

4. CRUD Operations — Create

CRUD stands for Create, Read, Update, Delete — the four fundamental operations any database must support. This unit covers all four in full depth, starting with Create.

4.1 insertOne() — Inserting a Single Document

```
db.students.insertOne({
  name: "Priya Sharma",
  age: 21,
  branch: "CSE",
  email: "priya@example.com"
})

// Return value on success:
// {
//   acknowledged: true,
//   insertedId: ObjectId('64f1a2b3c4d5e6f7a8b9c0d1')
// }
```

- If you do not supply an `_id` field, MongoDB auto-generates one as an `ObjectId` and returns it in `insertedId`.
- If you DO supply your own `_id`, MongoDB uses it as-is — but it must be unique within the collection, or the insert fails with a duplicate key error.

4.2 insertMany() — Inserting Multiple Documents

```
db.students.insertMany([
  { name: "Arjun Mehta", age: 22, branch: "ECE" },
  { name: "Sneha Rao", age: 20, branch: "CSE" },
  { name: "Karan Patel", age: 23, branch: "ME" }
])

// Return value:
// {
//   acknowledged: true,
//   insertedIds: {
//     '0': ObjectId('...'), '1': ObjectId('...'), '2': ObjectId('...')
//   }
// }
```

By default, `insertMany()` inserts documents in ORDER, and stops immediately on the first error (ordered inserts). You can change this with the `ordered` option:

```
// Unordered insert: MongoDB attempts ALL documents even if some fail,
// reporting all successes and all failures together at the end.
db.students.insertMany(
  [ { name: "Riya" }, { name: "Zoya" } ],
```

```
{ ordered: false }
)
```

NOTE: Ordered vs. unordered is a common exam/interview distinction: ordered=true (the default) is safer for maintaining sequence-dependent data but slower on batch failures (it halts early); ordered=false is faster for bulk-loading independent records where you want maximum throughput and can handle partial failures afterward.

4.3 Write Concern on Insert Operations

As covered in Unit I Section 3.5, every write operation (including inserts) can specify a writeConcern controlling how many replica set members must acknowledge the write before it's considered successful.

```
// Insert with the default write concern (acknowledged by primary only)
db.students.insertOne({ name: "Default WC" })

// Insert requiring acknowledgment from a MAJORITY of replica set members
// -- safer, slightly slower, recommended for important/financial data
db.students.insertOne(
  { name: "Majority WC", critical: true },
  { writeConcern: { w: "majority", wtimeout: 5000 } }
)

// wtimeout: 5000 -> give up waiting after 5 seconds and return an error
// rather than hanging indefinitely if the majority is unreachable
```

4.4 Document Validation on Insert

If the target collection has schema validation rules attached (Section 3.3), every insert is checked against them BEFORE being accepted.

```
// Assuming 'students' has the validator from Section 3.3 requiring name + email:

// This SUCCEEDS -- satisfies the schema
db.students.insertOne({ name: "Valid Student", email: "valid@example.com" })

// This FAILS -- missing required 'email' field
db.students.insertOne({ name: "Invalid Student" })
// MongoServerError: Document failed validation
```

NOTE: Document validation in MongoDB is intentionally OPTIONAL and OPT-IN per collection — this is the deliberate middle ground between total schema freedom and SQL's rigid schema-on-write, letting a team enforce structure only where it truly matters (e.g., a 'payments' collection) while leaving other collections (e.g., an 'eventLogs' collection) fully flexible.

4.5 Common Insert Errors & How to Read Them

Error	Cause	Fix
E11000 duplicate key error	Inserting a document with an <code>_id</code> (or another uniquely-indexed field) that already exists.	Use a different/omitted <code>_id</code> , or use <code>updateOne</code> with <code>upsert:true</code> if you meant to update-or-insert.
Document failed validation	The document doesn't satisfy the collection's <code>\$jsonSchema</code> validator.	Check <code>validationAction</code> — if 'error', fix the document's fields to match required types/patterns.
BSONObj size ... exceeds maximum	A single document exceeds the 16MB BSON size limit.	Split large content into multiple documents/collections, or use GridFS for large binary files.

5. CRUD Operations — Read

5.1 find() vs. findOne()

```
// find() returns a CURSOR pointing to ALL matching documents (lazy-evaluated)
db.students.find({ branch: "CSE" })

// findOne() returns a single document directly (the FIRST match), or null if none
match
db.students.findOne({ branch: "CSE" })

// find() with no filter returns ALL documents in the collection
db.students.find()

// .pretty() formats cursor output with indentation for readability
db.students.find().pretty()
```

NOTE: A cursor is NOT the result set itself — it's a pointer/handle that lazily fetches documents from the server in batches as you iterate it. This is why mongosh only prints the first 20 results by default and shows 'Type "it" for more' — it hasn't fetched the rest yet, saving memory and network bandwidth for large result sets.

5.2 Query Filters and Selection Operators

The first argument to find()/findOne() is a QUERY FILTER — a document describing which records to match. Simple equality is written directly; more complex conditions use operators (always prefixed with \$).

Operator	Meaning	Example
\$eq	Equals (default if no operator given)	{ branch: { \$eq: "CSE" } } // same as { branch: "CSE" }
\$ne	Not equal to	{ branch: { \$ne: "CSE" } }
\$gt / \$gte	Greater than / greater than or equal	{ age: { \$gt: 21 } }
\$lt / \$lte	Less than / less than or equal	{ cgpa: { \$lte: 7.5 } }
\$in	Matches ANY value in a given array	{ branch: { \$in: ["CSE", "ECE"] } }
\$nin	Matches NONE of the values in a given array	{ branch: { \$nin: ["ME", "CE"] } }
\$and	ALL sub-conditions must be true (implicit when listing multiple fields)	{ \$and: [{ age: { \$gt: 20 } }, { branch: "CSE" }] }
\$or	AT LEAST ONE sub-condition must be true	{ \$or: [{ branch: "CSE" }, { cgpa: { \$gt: 9 } }] }
\$not	Negates a condition	{ age: { \$not: { \$gt: 25 } } }
\$exists	Matches documents where a field is present (or absent)	{ scholarship: { \$exists: true } }
\$elemMatch	Matches an ARRAY field where at least one element satisfies ALL given conditions	{ grades: { \$elemMatch: { course: "CSE494", grade: "A" } } }

Operator	Meaning	Example
\$regex	Pattern-matches string fields (supports full regex syntax)	{ name: { \$regex: "^Pri", \$options: "i" } }

Worked Examples

```
// Students older than 21 in the CSE branch
db.students.find({ age: { $gt: 21 }, branch: "CSE" })

// Students in EITHER CSE or ECE
db.students.find({ branch: { $in: ["CSE", "ECE"] } })

// Students who are in CSE AND have cgpa above 8
db.students.find({ $and: [ { branch: "CSE" }, { cgpa: { $gt: 8 } } ] })
// Equivalent shorthand (implicit AND when listing multiple top-level fields):
db.students.find({ branch: "CSE", cgpa: { $gt: 8 } })

// Students who scored an 'A' in CSE494 specifically (inside an array of embedded docs)
db.students.find({ grades: { $elemMatch: { course: "CSE494", grade: "A" } } })
```

PRODUCTION NOTE: Without \$elemMatch, a query like { "grades.course": "CSE494", "grades.grade": "A" } can incorrectly match a document where ONE array element has course:'CSE494' with a DIFFERENT grade, and a SEPARATE element happens to have grade:'A' for a different course — because MongoDB checks each condition against the array independently, not against the SAME element. \$elemMatch forces both conditions onto the SAME array element, which is almost always what you actually intend.

5.3 Projection — Selecting Which Fields to Return

The second argument to find() is a PROJECTION — controlling which fields are included or excluded in the returned documents (reducing network payload when you don't need the whole document).

```
// Include ONLY name and branch (_id is included by default unless excluded)
db.students.find({}, { name: 1, branch: 1 })

// Explicitly exclude _id while including specific fields
db.students.find({}, { name: 1, branch: 1, _id: 0 })

// EXCLUDE specific fields, include everything else
db.students.find({}, { email: 0, address: 0 })
```

PRODUCTION NOTE: You cannot mix inclusion (field: 1) and exclusion (field: 0) in the same projection, EXCEPT for _id, which can always be explicitly excluded (_id: 0) alongside an inclusion projection. Mixing other 1s and 0s together throws an error.

5.4 Cursor Methods: limit(), skip(), sort()


```
// Return only the first 5 matching documents
db.students.find().limit(5)

// Skip the first 10 documents (commonly combined with limit for pagination)
db.students.find().skip(10).limit(5) // 'page 3' if page size is 5

// Sort ascending (1) or descending (-1) by a field
db.students.find().sort({ cgpa: -1 }) // highest CGPA first
db.students.find().sort({ branch: 1, age: -1 }) // sort by branch A-Z, then age high-
to-low within each branch

// Methods can be CHAINED together -- order of chaining in code does NOT
// change execution order; MongoDB always applies sort -> skip -> limit internally
db.students.find({ branch: "CSE" }).sort({ cgpa: -1 }).skip(0).limit(3)

// Count how many documents match a filter (without fetching them)
db.students.countDocuments({ branch: "CSE" })
```

PRODUCTION NOTE: `skip()` becomes progressively SLOWER on large collections, because MongoDB must still walk through and discard all skipped documents internally — it doesn't 'jump' directly to an offset. For deep pagination on large datasets, range-based pagination (e.g., filtering on `_id: { $gt: lastSeenId }`) is the recommended production pattern instead of a large `skip()` value.

5.5 Querying Inside Embedded Documents and Arrays

```
// Dot notation reaches into a nested (embedded) sub-document field
db.students.find({ "address.city": "Ludhiana" })

// Querying a plain array field for a specific value (matches if the array CONTAINS
it)
db.students.find({ courses: "CSE494" })

// Querying an array field by POSITION (index 0 specifically)
db.students.find({ "courses.0": "CSE494" })

// $size -- match arrays of an exact length
db.students.find({ courses: { $size: 2 } })

// $all -- match arrays that contain ALL given values (in any order)
db.students.find({ courses: { $all: ["CSE494", "CSE401"] } })
```

5.6 Going Deeper: How MongoDB Actually Evaluates a Query Internally

Understanding the internal evaluation order helps explain both performance and some surprising query behaviors. When you call `find(filter).sort(...).skip(...).limit(...)`, MongoDB does NOT execute these in the order you wrote them in code — the query planner decides the most efficient physical execution order, which is typically:

1. Query Planning: MongoDB examines the filter and available indexes, and chooses a 'winning plan' — either an IXSCAN (using an index) or a COLLSCAN (scanning every document).

2. Filtering: Documents are matched against the filter conditions (either via the index directly, or by examining each document during a collection scan).
3. Sorting: If an index doesn't already provide the results in sorted order, MongoDB performs an in-memory sort of the matched documents (limited by a default 100MB memory cap — an unindexed sort exceeding this throws an error unless `allowDiskUse` is used, a detail relevant in later units).
4. Skipping: The specified number of leading documents are discarded.
5. Limiting: Only the requested number of remaining documents are returned to the client.

NOTE: Why this matters practically: if a query can use an index that ALREADY stores data in the sorted order you requested (e.g., an index on `cgpa` when you sort by `cgpa`), MongoDB can skip the expensive in-memory sort step entirely — this is one of the most impactful, real-world reasons to design indexes around your application's actual `sort()` calls, not just its filter conditions.

6. CRUD Operations — Update

6.1 updateOne() vs. updateMany() vs. replaceOne()

Method	Affects	Behavior
updateOne()	The FIRST document matching the filter	Modifies only the specified fields (via update operators), leaving all other fields untouched.
updateMany()	ALL documents matching the filter	Same field-level modification behavior as updateOne, applied to every match.
replaceOne()	The FIRST document matching the filter	REPLACES the entire document body (except <code>_id</code>) with a brand-new document you supply — any field not in the replacement is LOST.

```
// updateOne: modify only the 'cgpa' field of one matching student
db.students.updateOne({ name: "Priya Sharma" }, { $set: { cgpa: 9.1 } })

// updateMany: give every CSE student a 'department' field
db.students.updateMany({ branch: "CSE" }, { $set: { department: "Computer Science" } })

// replaceOne: completely replace the document body (all old fields except _id are gone)
db.students.replaceOne(
  { name: "Priya Sharma" },
  { name: "Priya Sharma", branch: "CSE", cgpa: 9.1 } // this is now the ENTIRE document
)
```

Every update method returns a result summarizing what happened:

```
// {
//   acknowledged: true,
//   matchedCount: 1,      -- how many documents matched the filter
//   modifiedCount: 1,     -- how many were ACTUALLY changed (0 if new value == old value)
//   upsertedId: null      -- populated only if upsert:true created a new document
// }
```

6.2 Update Operators

Operator	Purpose	Example
\$set	Set a field to a specific value (creates the field if it doesn't exist)	{ \$set: { cgpa: 9.1 } }
\$unset	Remove a field entirely from the document	{ \$unset: { middleName: "" } }
\$inc	Increment (or decrement, with a negative value) a numeric field atomically	{ \$inc: { loginCount: 1 } }

Operator	Purpose	Example
\$mul	Multiply a numeric field by a given value	{ \$mul: { price: 1.1 } } // 10% price increase
\$rename	Rename a field	{ \$rename: { oldName: "newName" } }
\$push	Append a value to an array field	{ \$push: { courses: "CSE402" } }
\$pull	Remove ALL array elements matching a given condition	{ \$pull: { courses: "CSE401" } }
\$addToSet	Append a value to an array ONLY IF it isn't already present (prevents duplicates)	{ \$addToSet: { courses: "CSE494" } }
\$pop	Remove the first (-1) or last (1) element of an array	{ \$pop: { courses: 1 } }

Worked Examples

```
// Atomically increment a view counter (safe under concurrent writes)
db.posts.updateOne({ _id: postId }, { $inc: { views: 1 } })

// Add a new course to a student's course list, avoiding duplicates
db.students.updateOne({ name: "Priya Sharma" }, { $addToSet: { courses: "CSE494" } })

// Remove one specific course from the list
db.students.updateOne({ name: "Priya Sharma" }, { $pull: { courses: "CSE401" } })

// Combine multiple operators in a SINGLE update call
db.students.updateOne(
  { name: "Priya Sharma" },
  {
    $set: { lastUpdated: new Date() },
    $inc: { loginCount: 1 },
    $push: { courses: "CSE402" }
  }
)
```

NOTE: Why does \$inc matter for concurrency? Because { \$inc: { views: 1 } } is executed ATOMICALLY server-side — if two clients increment simultaneously, both increments are safely applied (no lost update). Doing the equivalent as 'read views, add 1 in application code, then \$set the new value' is NOT safe under concurrency — two simultaneous reads could both read the same starting value and one increment would be silently lost.

6.3 Upsert — Update-or-Insert

Setting upsert: true tells MongoDB: 'update the matching document if one exists; otherwise, INSERT a new document using the filter + update criteria combined.' This is extremely useful for idempotent write patterns (e.g., syncing external data, incrementing per-user counters that might not exist yet).

```
// If a document with email 'new@example.com' exists, update its 'lastLogin' field.
// If it does NOT exist, INSERT a brand-new document combining the filter + $set fields.
```

```
db.students.updateOne(
  { email: "new@example.com" },
  { $set: { lastLogin: new Date() }, $setOnInsert: { branch: "Undeclared" } },
  { upsert: true }
)
```

NOTE: \$setOnInsert sets fields ONLY when the upsert results in an INSERT (i.e., no matching document existed) — it is ignored if an existing document was updated instead. This is the standard way to set 'default' values that should only apply to brand-new records, e.g., a default branch or a createdAt timestamp.

6.4 Going Deeper: What 'Atomic' Actually Guarantees (and What It Doesn't)

The term 'atomic' gets used loosely, so it's worth being precise about MongoDB's actual guarantee: every update operation (updateOne, updateMany applied per-matched-document, \$inc, \$push, etc.) is atomic AT THE SINGLE-DOCUMENT LEVEL. This means all the modifications specified within ONE update call, applied to ONE document, either fully succeed or fully fail together — another client can never observe a document in a half-updated state.

- **Guaranteed:** `db.students.updateOne({_id: x}, {$set: {a: 1}, $inc: {b: 1}})` — both the \$set and \$inc happen together, atomically, as one indivisible operation on that single document.
- **NOT automatically guaranteed:** `updateMany({branch: 'CSE'}, {$set: {flag: true}})` modifying 500 matching documents — each INDIVIDUAL document's update is atomic, but the batch of 500 updates as a WHOLE is not one atomic transaction. If the server crashes partway through, some of the 500 documents may be updated and others not.

NOTE: If you need TRUE multi-document atomicity (all-or-nothing across many documents/collections), MongoDB's multi-document ACID transactions (introduced in Unit I, Section 2.3) are the correct tool — but they carry additional performance overhead and are reserved for genuinely cross-document consistency needs, not routine bulk updates like the CSE-branch example above.

7. CRUD Operations — Delete

7.1 deleteOne() vs. deleteMany()

```
// Delete the FIRST document matching the filter
db.students.deleteOne({ name: "Karan Patel" })

// Delete ALL documents matching the filter
db.students.deleteMany({ branch: "ME" })

// DANGER: an empty filter {} matches EVERY document in the collection
db.students.deleteMany({}) // wipes the ENTIRE collection's data (but keeps the
collection + indexes)

// Return value:
// { acknowledged: true, deletedCount: 3 }
```

7.2 Write Concern on Delete Operations

```
// Delete with majority write concern for critical data
db.payments.deleteOne(
  { _id: paymentId },
  { writeConcern: { w: "majority", wtimeout: 5000 } }
)
```

7.3 Safety Best Practices Before Deleting

6. ALWAYS run the equivalent find() with your intended filter FIRST, and visually confirm the result set before converting it to a delete. E.g., run db.students.find({ branch: "ME" }) before running db.students.deleteMany({ branch: "ME" }).
7. Count first: use countDocuments() with the same filter to sanity-check how many records will be affected before committing to the delete — an unexpectedly large count is often a sign the filter is wrong.
8. Prefer soft deletes for important business data: instead of physically deleting, set a flag like { \$set: { isDeleted: true, deletedAt: new Date() } } — this preserves an audit trail and allows recovery, at the cost of needing to filter isDeleted:false in normal queries.
9. Take a backup (mongodump) before any bulk delete in an environment that matters, and double-check with `db` which database you're currently connected to.
10. Use role-based access control (RBAC) in production so that only specifically authorized roles can execute deleteMany() or dropDatabase() against production clusters.

```
// SAFE WORKFLOW EXAMPLE

// Step 1: preview exactly what will be affected
db.students.find({ branch: "ME" })
```

```
// Step 2: confirm the count matches expectations
db.students.countDocuments({ branch: "ME" })
// -> 3   (expected? proceed. unexpected? STOP and re-check the filter)

// Step 3: only now, run the actual delete
db.students.deleteMany({ branch: "ME" })
```

PRODUCTION NOTE: There is NO undo for deleteOne()/deleteMany() in MongoDB itself. Unlike some SQL clients that wrap statements in an implicit transaction you can roll back, mongosh commands execute immediately and permanently against the server. Building the 'preview, then confirm, then act' habit is one of the most valuable production habits you can develop in this unit.

8. Comic-Style Scenario: The BSON Postal Service

To make the journey of a write operation intuitive and memorable, imagine MongoDB as a highly organized postal service that receives your JSON 'letters,' translates them into efficient binary 'packages,' and files them precisely for instant future retrieval.

PANEL 1 — You Write a Letter (the JSON Document)

You (the application): "I'd like to send this letter to the 'students' filing cabinet."

```
{ name: "Priya Sharma", branch: "CSE", age: 21 }
```

 (Your document is plain, human-readable JSON at this point — easy for you to write, but inefficient for the postal service to file quickly.)

PANEL 2 — The BSON Sorting Office Repackages It

Postal Clerk (BSON Encoder): "Let me convert this into a binary package — each field gets a type tag and a length prefix, so future clerks never have to re-read the whole letter to find one field."

```
*stamps*: [type:string][len:4]"name"... [type:string][len:16]"Priya Sharma"...
```

 (This is the JSON-to-BSON encoding step — happening automatically, invisibly, the instant your driver/shell sends the insert command.)

PANEL 3 — The Package Gets a Tracking Number (_id / ObjectId)

Postal Clerk: "Every package needs a unique tracking number before filing."

```
*stamps*: ObjectId("64f1a2b3c4d5e6f7a8b9c0d1")
```

 (No _id was given, so MongoDB mints one: 4 bytes timestamp + 5 bytes machine ID + 3 bytes counter = a fast, coordinator-free unique ID.)

PANEL 4 — Filed in the Correct Cabinet (Collection) and Drawer (WiredTiger)

Postal Clerk: "This package belongs in the 'students' cabinet, inside the 'college' warehouse." *files it into a WiredTiger data file on disk*

 (The document is now durably stored — but first logged in the journal, so even a power cut a millisecond later won't lose it.)

PANEL 5 — A Lookup Map Is Updated (Indexes)

Index Clerk: "I keep a separate alphabetical directory of every _id so I never have to search the whole warehouse for one package."

```
*updates the _id index B-tree with a pointer to the new package's location*
```

 (This is why querying by _id is always fast — the index points

directly to the document's location, no full-collection scan needed.)

PANEL 6 — Delivery Confirmation Returns to You

Postal Clerk: "Package filed successfully! Here's your receipt."

```
{ acknowledged: true, insertedId: ObjectId('64f1a2b3c4d5e6f7a8b9c0d1') }
```

 (The whole journey — JSON to BSON, ID generation, journaling, disk storage, and index update — happens in milliseconds, and this receipt is the ONLY part of the process visible to your application code.)

9. Data Modeling, Indexes & Query Optimization Basics

NOTE: Full schema design strategy (embedding vs. referencing in depth) is the dedicated subject of Unit III, and the full Aggregation Framework belongs to Unit IV. This section covers only the FOUNDATIONAL indexing/modeling concepts that CO2 requires at the Unit II level — enough to write efficient CRUD code today.

9.1 Why Indexes Exist — The Core Problem

Without an index, MongoDB must perform a COLLECTION SCAN (COLLSCAN) to satisfy a query — examining every single document in the collection, one by one, checking whether it matches the filter. On a collection with a few dozen documents this is instant; on a collection with 50 million documents, this could take many seconds or minutes per query.

An index is a separate, ordered data structure (a B-tree) that stores a sorted mapping from field values to document locations — similar to the index at the back of a textbook, which lets you jump directly to a page rather than reading the entire book to find a topic.

WITHOUT AN INDEX (Collection Scan)	WITH AN INDEX (B-Tree lookup)
-----	-----
Query: <code>find({ name: "Priya" })</code>	Query: <code>find({ name: "Priya" })</code>
doc1 -> check name... no match	Index B-Tree on 'name':
doc2 -> check name... no match	[Arjun]-->loc42
doc3 -> check name... no match	[Karan]-->loc07
doc4 -> check name... MATCH!	[Priya]-->loc19 <-- direct jump!
doc5 -> check name... no match	[Sneha]-->loc55
... (continues through ALL documents)	
Cost: $O(n)$ -- scales with collection size	Cost: $O(\log n)$ -- scales with tree depth

9.2 Creating a Single-Field Index

```
// Create an ascending index on the 'name' field
db.students.createIndex({ name: 1 })

// Create a descending index on 'cgpa' (direction matters for compound indexes
// and for sort efficiency, though it rarely matters for a single-field index alone)
db.students.createIndex({ cgpa: -1 })

// List all indexes on a collection
db.students.getIndexes()

// Every collection ALWAYS has one index by default: the _id index,
// automatically created and never removable -- this is why querying
// by _id is always fast even with zero manual index creation.
```

9.3 Creating a Compound (Multi-Field) Index

A compound index covers MULTIPLE fields in a specified order — useful when queries commonly filter/sort on more than one field together.

```
// Compound index on branch (ascending) then cgpa (descending)
db.students.createIndex({ branch: 1, cgpa: -1 })

// This index efficiently supports queries filtering on 'branch' alone,
// OR on 'branch' AND 'cgpa' together, OR sorting by cgpa WITHIN a branch filter:
db.students.find({ branch: "CSE" })
db.students.find({ branch: "CSE", cgpa: { $gt: 8 } })
db.students.find({ branch: "CSE" }).sort({ cgpa: -1 })

// It does NOT efficiently support a query filtering on 'cgpa' ALONE,
// because 'branch' is the LEADING (first) field in the index --
// this is called the 'ESR rule' foundation, expanded fully in Unit IV.
```

NOTE: Field order in a compound index matters enormously — an index on { branch: 1, cgpa: -1 } is fundamentally different from one on { cgpa: -1, branch: 1 }, in the same way a phone book sorted by (last name, first name) can't efficiently answer 'find everyone named John' without scanning the whole book. Always put the field(s) used for EQUALITY filters first, and fields used for RANGE filters or SORTING later.

9.4 Write Performance Implications of Indexes

Indexes are not free — every index you create must be updated on EVERY insert, update, and delete that touches the indexed field(s), because the B-tree structure must stay in sync with the actual data.

Aspect	Effect of adding an index
Read performance	Dramatically faster for queries that can use the index (avoids full collection scans).
Write performance	Slightly slower — every write must also update each relevant index's B-tree structure.
Storage	Each index consumes additional disk space and RAM (ideally, indexes should fit in the WiredTiger cache for best performance).
Maintenance	More indexes = more overhead to maintain; unused indexes should be identified and dropped.

PRODUCTION NOTE: A common production anti-pattern is 'index everything just in case.' Each unnecessary index silently taxes every write operation on the collection forever, while providing no benefit unless a query actually uses it. The standard practice is to create indexes deliberately, based on your application's ACTUAL query patterns — not preemptively on every field.

9.5 Reading a Basic Execution Plan: `.explain("executionStats")`

The `explain()` method reveals HOW MongoDB actually executed a given query — whether it used an index (IXSCAN) or fell back to a full collection scan (COLLSCAN), how many documents it examined, and how long it took. This is your primary diagnostic tool for query optimization.

```
// Attach .explain("executionStats") to see detailed execution metrics
```

```
db.students.find({ branch: "CSE" }).explain("executionStats")
```

Key fields to read in the output:

Field (inside executionStats)	Meaning
stage (in winningPlan)	"COLLSCAN" = full collection scan (no usable index); "IXSCAN" = an index was used.
nReturned	How many documents the query actually returned/matched.
totalDocsExamined	How many documents MongoDB had to physically look at. Ideally close to nReturned.
totalKeysExamined	How many index entries were scanned (relevant only when an index is used).
executionTimeMillis	Total time the query took to execute, in milliseconds.

```
// BEFORE creating an index -- notice totalDocsExamined is large,
// and stage shows COLLSCAN
db.students.find({ branch: "CSE" }).explain("executionStats")
// winningPlan.stage: "COLLSCAN"
// executionStats.totalDocsExamined: 50000
// executionStats.nReturned: 8000

// Now create an index on the filtered field
db.students.createIndex({ branch: 1 })

// AFTER creating the index -- re-run the SAME query
db.students.find({ branch: "CSE" }).explain("executionStats")
// winningPlan.stage: "IXSCAN"
// executionStats.totalDocsExamined: 8000  <- now matches nReturned almost exactly!
// executionStats.executionTimeMillis: dramatically lower
```

NOTE: Rule of thumb for a 'healthy' query: totalDocsExamined should be as close as possible to nReturned. A huge gap between them (examining far more documents than you actually needed) is the classic symptom that a useful index is missing — this single diagnostic habit is one of the most practical, job-relevant skills in this entire unit.

10. Big Unified Example: University Course Portal (End-to-End)

This single running example walks through EVERY concept covered in this unit, in order, against one realistic scenario: building the backend data layer for a University Course Portal. Follow it top to bottom in your own mongosh session — every command is copy-pasteable and builds on the previous one.

Step 1 — Set Up the Database and Explicit Collection with Validation

```
use university_portal

db.createCollection("students", {
  validator: {
    $jsonSchema: {
      bsonType: "object",
      required: ["name", "email", "branch"],
      properties: {
        name: { bsonType: "string" },
        email: { bsonType: "string", pattern: "^.+@.+$" },
        branch: { bsonType: "string" },
        cgpa: { bsonType: ["double", "int"], minimum: 0, maximum: 10 }
      }
    }
  },
  validationAction: "error"
})
```

Step 2 — Create: Insert Students (insertOne + insertMany)

```
db.students.insertOne({
  name: "Priya Sharma",
  email: "priya@example.com",
  branch: "CSE",
  age: 21,
  cgpa: 8.9,
  enrolledOn: ISODate("2024-08-01"),
  courses: ["CSE494", "CSE401"],
  address: { city: "Ludhiana", state: "Punjab" }
})

db.students.insertMany([
  {
    name: "Arjun Mehta", email: "arjun@example.com", branch: "ECE",
    age: 22, cgpa: 7.8, enrolledOn: ISODate("2023-08-01"),
    courses: ["ECE301"], address: { city: "Chandigarh", state: "Punjab" }
  },
  {
    name: "Sneha Rao", email: "sneha@example.com", branch: "CSE",
```

```

    age: 20, cgpa: 9.4, enrolledOn: ISODate("2025-08-01"),
    courses: ["CSE494"], address: { city: "Pune", state: "Maharashtra" }
  }, { ordered: false })

```

Step 3 — Read: Query the Data We Just Inserted

```

// All CSE students, sorted by cgpa descending, name+cgpa only
db.students.find({ branch: "CSE" }, { name: 1, cgpa: 1, _id: 0 }).sort({ cgpa: -1 })

// Students enrolled in CSE494 specifically
db.students.find({ courses: "CSE494" })

// Students in Punjab with cgpa above 8
db.students.find({ "address.state": "Punjab", cgpa: { $gt: 8 } })

// Top scorer only
db.students.findOne({}, {}, { sort: { cgpa: -1 } })
// (equivalent alternative: db.students.find().sort({cgpa:-1}).limit(1))

```

Step 4 — Update: Modify Records as the Semester Progresses

```

// Priya enrolls in a new course, avoiding duplicates
db.students.updateOne({ name: "Priya Sharma" }, { $addToSet: { courses: "CSE402" } })

// Sneha's CGPA improves after results are published
db.students.updateOne({ name: "Sneha Rao" }, { $set: { cgpa: 9.6 } })

// Increment a login counter atomically (field created automatically on first use)
db.students.updateOne({ name: "Arjun Mehta" }, { $inc: { loginCount: 1 } })

// Give ALL CSE students a scholarship-eligible flag
db.students.updateMany({ branch: "CSE", cgpa: { $gte: 9 } }, { $set: {
  scholarshipEligible: true } })

// Upsert: log today's attendance for a student who may not have a record yet
db.attendance.updateOne(
  { studentEmail: "priya@example.com", date: "2026-08-13" },
  { $set: { present: true }, $setOnInsert: { markedBy: "system" } },
  { upsert: true }
)

```

Step 5 — Index: Speed Up the Portal's Most Common Query

```

// The portal frequently filters by branch and sorts by cgpa -- add a compound index
db.students.createIndex({ branch: 1, cgpa: -1 })

```

```
// Verify the index is actually being used now
db.students.find({ branch: "CSE" }).sort({ cgpa: -1 }).explain("executionStats")
// winningPlan.stage should now show IXSCAN instead of COLLSCAN
```

Step 6 — Delete: End-of-Semester Cleanup (Safely)

```
// Step 6a: PREVIEW before deleting -- find graduated students to be archived
db.students.find({ branch: "ME" })

// Step 6b: confirm count
db.students.countDocuments({ branch: "ME" })

// Step 6c: soft-delete preferred in production -- mark rather than physically remove
db.students.updateMany({ branch: "ME" }, { $set: { isArchived: true, archivedAt: new Date() } })

// Step 6d: (only if truly permanent removal is required) hard delete
db.students.deleteMany({ branch: "ME", isArchived: true })
```

Step 7 — Inspect the Final State

```
db.students.countDocuments()
db.students.getIndexes()
db.stats()
show collections
```

NOTE: Notice how every concept from Sections 1–9 appears here in a natural, realistic order: schema validation at creation, insertOne/insertMany for onboarding data, rich queries with projections and sorting for the portal UI, targeted updates as data changes over time, a compound index once a query pattern becomes frequent, and a careful preview-then-delete workflow at cleanup time. This is the exact rhythm of real backend development with MongoDB.

11. Practical Exercises (with Solutions)

Exercise 1 — Library Management CRUD (Basic–Intermediate)

Task: Create a database library. Create a collection books with schema validation requiring title (string), author (string), and copies (int, minimum 0). Insert 4 books. Find all books by a specific author. Update one book to reduce its copies by 1 (simulating a checkout) using \$inc. Delete any book with 0 copies remaining.

Solution:

```
use library

db.createCollection("books", {
  validator: {
    $jsonSchema: {
      bsonType: "object",
      required: ["title", "author", "copies"],
      properties: {
        title: { bsonType: "string" },
        author: { bsonType: "string" },
        copies: { bsonType: "int", minimum: 0 }
      }
    }
  }
})

db.books.insertMany([
  { title: "Clean Code", author: "Robert Martin", copies: 3 },
  { title: "The Pragmatic Programmer", author: "David Thomas", copies: 2 },
  { title: "Refactoring", author: "Martin Fowler", copies: 1 },
  { title: "Domain-Driven Design", author: "Martin Fowler", copies: 0 }
])

db.books.find({ author: "Martin Fowler" })

// Simulate a checkout of "Refactoring"
db.books.updateOne({ title: "Refactoring" }, { $inc: { copies: -1 } })

// Clean up books with zero copies remaining
db.books.deleteMany({ copies: 0 })
```

Exercise 2 — Query Operators Practice (Intermediate)

Task: Using the students collection from Section 10, write single queries for: (a) students aged between 20 and 22 inclusive; (b) students in CSE or ECE with cgpa above 8; (c) students who do NOT have a scholarshipEligible field at all; (d) the second-highest CGPA student (skip 1, limit 1, sorted descending).

Solution:


```
// (a) age between 20 and 22 inclusive
db.students.find({ age: { $gte: 20, $lte: 22 } })

// (b) CSE or ECE, cgpa above 8
db.students.find({ branch: { $in: ["CSE", "ECE"] }, cgpa: { $gt: 8 } })

// (c) no scholarshipEligible field present
db.students.find({ scholarshipEligible: { $exists: false } })

// (d) second-highest CGPA
db.students.find().sort({ cgpa: -1 }).skip(1).limit(1)
```

Exercise 3 — Indexing & Query Optimization Audit (Advanced)

Task: On the students collection, run `.explain("executionStats")` for a query filtering on email (used for login lookups) BEFORE any index exists. Observe the COLLSCAN. Create an appropriate index. Re-run `explain()` and confirm IXSCAN is now used, and that `totalDocsExamined` has dropped to match `nReturned`.

Solution:

```
// BEFORE: observe COLLSCAN
db.students.find({ email: "priya@example.com" }).explain("executionStats")
// winningPlan.stage: "COLLSCAN"

// Since email is used for point lookups AND should be unique per student,
// create a UNIQUE index -- this both speeds up queries AND enforces
// data integrity (no two students can share an email going forward)
db.students.createIndex({ email: 1 }, { unique: true })

// AFTER: confirm IXSCAN and tight totalDocsExamined
db.students.find({ email: "priya@example.com" }).explain("executionStats")
// winningPlan.stage: "IXSCAN"
// executionStats.totalDocsExamined: 1
// executionStats.nReturned: 1
```

NOTE: This exercise demonstrates a subtle but important point: indexes aren't just a performance tool — the `unique: true` option turns an index into a DATA INTEGRITY constraint as well, rejecting any future insert/update that would create a duplicate email, similar to a UNIQUE constraint in SQL.

12. Self-Assessment: Conceptual Review Questions (with Model Answers)

Attempt each question yourself in writing before reading the model answer.

Question 1

Explain why MongoDB uses BSON instead of storing documents as plain text JSON, citing at least two concrete technical advantages.

ANSWER: Model Answer:

BSON (Binary JSON) is a binary-encoded superset of JSON designed specifically to address weaknesses plain-text JSON has as a database storage/transport format. Two concrete advantages: (1) BSON is length-prefixed and binary-encoded, so the storage engine can jump directly to a specific field or skip over an entire embedded sub-document without scanning every character — this makes traversal and partial-field access dramatically faster than re-parsing text JSON on every read. (2) BSON supports richer, more precise data types than JSON's single generic 'number' type — including a native Date type (avoiding slow string-based date parsing), Decimal128 for exact financial-grade precision, and a dedicated ObjectId type — eliminating ambiguity and repeated type-conversion overhead that plain JSON would otherwise require in application code.

Question 2

A collection has 500,000 documents. A query filtering on a non-indexed field currently takes 4 seconds and shows totalDocsExamined: 500000 in its execution plan. After creating an index on that field, explain exactly what changes internally, and what you would now expect to see in the new execution plan.

ANSWER: Model Answer:

Before the index, MongoDB has no ordered structure pointing directly to matching documents for that field, so it must perform a COLLSCAN (collection scan) — reading every one of the 500,000 documents sequentially and checking each against the filter, which is why totalDocsExamined equals the full collection size regardless of how few documents actually match.

After creating an index on that field, MongoDB builds a separate B-tree structure mapping sorted field values directly to document locations. The SAME query can now use an IXSCAN (index scan): it walks only the relevant portion of the B-tree to find matching values, then fetches only those specific documents by location — rather than reading the entire collection. The new execution plan should show winningPlan.stage as "IXSCAN", a totalDocsExamined value close to nReturned (only the actual matches, not all 500,000), and a substantially lower executionTimeMillis.

Question 3

A junior developer writes: `db.orders.updateOne({ _id: orderId }, { quantity: 5 })` intending to change the 'quantity' field to 5, but the entire document is unexpectedly wiped except for `_id` and `quantity`. Explain exactly what went wrong and how to fix it.

ANSWER: Model Answer:

The developer forgot to wrap the change in an UPDATE OPERATOR (such as \$set). Without an operator, MongoDB interprets the second argument as a REPLACEMENT document for `updateOne()` — meaning it replaces the ENTIRE document body with exactly `{ quantity: 5 }` (keeping only `_id`, which cannot be replaced), silently discarding every other field the document previously had.

The fix is to use the \$set operator to modify only the intended field while leaving all others untouched: `db.orders.updateOne({ _id: orderId }, { $set: { quantity: 5 } })`. As a broader lesson: `updateOne()/updateMany()` require an update-operator-wrapped document to perform a partial modification; passing a plain field-value document (no \$ operators) is only valid for `replaceOne()`, where full replacement is the explicit intent.

13. Industry Interview & Viva Question Bank (30 Questions)

Strictly scoped to Unit II syllabus content. Each answer follows the definition-first, scoring-oriented format established in Unit I.

Q1. What are the three levels of MongoDB's data hierarchy, from largest to smallest?

SCORING ANSWER: Database (top-level container) → Collection (a named group of documents within a database) → Document (a single BSON record within a collection).

Q2. Map 'Table' and 'Row' from SQL to their closest MongoDB equivalents, and name one key difference for each.

SCORING ANSWER: Table → Collection (difference: a collection has no fixed schema; documents inside can vary in structure). Row → Document (difference: a document is a rich, nested BSON object that can contain arrays and sub-documents, not just flat scalar values).

Q3. What is the maximum size of a single MongoDB document, and what mechanism handles larger binary content?

SCORING ANSWER: 16MB is the hard maximum BSON document size. Larger binary content (e.g., large files/images) is handled using GridFS, which splits data into smaller chunks stored across multiple documents.

Q4. What does BSON stand for, and why does MongoDB use it instead of plain-text JSON?

SCORING ANSWER: BSON stands for Binary JSON. MongoDB uses it because it's length-prefixed and binary-encoded (faster to traverse/parse than scanning text), and it supports richer, unambiguous data types (Date, Decimal128, ObjectId, binary data) that plain JSON lacks natively.

Q5. Describe the three components of a default MongoDB ObjectId.

SCORING ANSWER: A 4-byte Unix timestamp (seconds since epoch), a 5-byte random value unique to the machine/process, and a 3-byte incrementing counter — totaling 12 bytes, generated without needing a central coordinating server.

Q6. Why are auto-generated ObjectIds roughly sortable by creation time?

SCORING ANSWER: Because the first 4 bytes of an ObjectId encode a Unix timestamp, sorting documents by `_id` approximates sorting them by insertion order, even though this isn't its primary purpose.

Q7. What happens when you run ``use mydb`` for a database that doesn't exist yet, and then immediately run ``show dbs``?

SCORING ANSWER: The database is NOT created immediately — MongoDB lazily creates databases only once actual data is written to them (e.g., via an insert or explicit `createCollection`). ``show dbs`` will not list 'mydb' until a write actually occurs.

Q8. What is the purpose of `db.createCollection()` when collections can already be created implicitly by inserting data?

SCORING ANSWER: Explicit creation is needed when you want to attach configuration options AT creation time that implicit creation doesn't support — most importantly schema validation rules (\$jsonSchema validator), capped-collection settings, and collation.

Q9. What is the difference between `validationAction: 'error'` and `validationAction: 'warn'`?

SCORING ANSWER: 'error' rejects any write that violates the schema validator outright. 'warn' logs a warning to the server log but still ALLOWS the write to proceed — typically used to test a new validation rule's impact before strictly enforcing it.

Q10. Explain the danger of `db.dropDatabase()` and one production safeguard against accidental use.

SCORING ANSWER: It permanently and irreversibly deletes the ENTIRE current database — all collections, documents, and indexes — with no confirmation prompt. A standard safeguard is role-based access control (RBAC), restricting this command to specific trusted admin roles rather than all developers.

Q11. What does `insertOne()` return on success, and what field contains the new document's identifier?

SCORING ANSWER: It returns an object like `{ acknowledged: true, insertedId: ObjectId(...) }`, where `insertedId` holds the identifier of the newly created document (auto-generated if none was supplied).

Q12. What is the difference in behavior between an `ORDERED` and an `UNORDERED` `insertMany()` call when one document in the batch fails validation?

SCORING ANSWER: Ordered (the default) stops immediately at the first failure, leaving any documents after it uninserted. Unordered attempts ALL documents regardless of earlier failures, reporting all successes and failures together at the end — useful for maximizing throughput on independent bulk records.

Q13. Name the query operator that safely matches a single array element satisfying MULTIPLE conditions simultaneously, and explain why it's needed.

SCORING ANSWER: `$elemMatch`. Without it, separate conditions on array-of-subdocument fields (e.g., `grades.course` and `grades.grade`) can each match DIFFERENT elements of the same array independently, producing false-positive matches. `$elemMatch` forces all listed conditions to be satisfied by the SAME array element.

Q14. What is a MongoDB cursor, and why doesn't `find()` return all matching documents immediately?

SCORING ANSWER: A cursor is a pointer/handle to the query's result set on the server, not the results themselves. `find()` is lazily evaluated — it fetches documents from the server in batches as the cursor is iterated, which conserves memory and network bandwidth for large result sets.

Q15. Why does `skip()` become progressively slower on large collections, and what's the recommended alternative for deep pagination?

SCORING ANSWER: `skip()` still requires MongoDB to walk through and discard all skipped documents internally rather than jumping directly to an offset, so its cost grows with the skip value. The recommended alternative is range-based ('keyset') pagination — filtering with a condition like `_id: { $gt: lastSeenId }` instead of an increasing skip value.

Q16. What is the difference between an inclusion projection and an exclusion projection, and what's the one exception to not mixing them?

SCORING ANSWER: Inclusion (`{ field: 1 }`) returns ONLY the specified fields; exclusion (`{ field: 0 }`) returns all fields EXCEPT the specified ones. You cannot mix 1s and 0s for different fields in the same projection, except that `_id` can always be explicitly excluded (`_id: 0`) even inside an otherwise-inclusion projection.

Q17. What is the functional difference between `updateOne()` and `replaceOne()`?

SCORING ANSWER: `updateOne()` (used with update operators like `$set`) modifies only the specified fields, leaving all other fields in the document untouched. `replaceOne()` replaces the ENTIRE document body (except `_id`) with a brand-new document, discarding any field not present in the replacement.

Q18. A developer runs `db.orders.updateOne({_id: id}, {status: 'shipped'})` and the whole document gets wiped except `_id` and `status`. What mistake was made?

SCORING ANSWER: The update argument was not wrapped in an update operator like `$set`. Without an operator, MongoDB treats the second argument as a full REPLACEMENT document (equivalent to `replaceOne` behavior), discarding every other existing field. The fix is `{ $set: { status: 'shipped' } }`.

Q19. Why is `$inc` considered safer than a 'read, increment in code, then `$set`' pattern under concurrent writes?

SCORING ANSWER: `$inc` executes atomically on the server — if two clients increment simultaneously, both increments are correctly applied with no lost update. A 'read then `$set`' pattern in application code can suffer a lost update if two clients read the same starting value before either writes back their result.

Q20. What is the difference between `$push` and `$addToSet`?

SCORING ANSWER: `$push` appends a value to an array regardless of whether it already exists, potentially creating duplicates. `$addToSet` appends a value ONLY IF it isn't already present in the array, preventing duplicates.

Q21. Explain what `upsert: true` does, and describe a realistic use case for it.

SCORING ANSWER: It tells an update operation to INSERT a new document (combining the filter and update criteria) if no document matches the given filter, instead of doing nothing. A realistic use case is idempotent syncing — e.g., recording a user's daily attendance, where the record should be created if it doesn't exist yet, or updated if it does, without needing separate insert/update logic.

Q22. What does `$setOnInsert` do, and when is it applied?

SCORING ANSWER: `$setOnInsert` sets specified fields ONLY when an upsert operation results in a brand-new INSERT (no existing document matched). It is silently ignored if the upsert instead updated an existing document — used for setting default values that should apply only to new records.

Q23. Why should you run `find()` and `countDocuments()` with your intended filter BEFORE running `deleteMany()` with that same filter?

SCORING ANSWER: Because delete operations in MongoDB are immediate and irreversible with no built-in undo. Previewing the exact documents and count that will be affected lets you catch an incorrect or overly broad filter before it causes accidental, permanent data loss.

Q24. What is a 'soft delete,' and why might a production system prefer it over `deleteMany()`?

SCORING ANSWER: A soft delete marks a document as deleted (e.g., { \$set: { isDeleted: true, deletedAt: new Date() } }) instead of physically removing it. Production systems often prefer this because it preserves an audit trail and allows recovery, at the cost of needing to filter isDeleted:false in normal application queries.

Q25. What problem do indexes solve, and what data structure do they use internally?

SCORING ANSWER: Indexes solve the problem of full collection scans (COLLSCAN) being slow on large collections, by providing a separate, ordered lookup structure. MongoDB indexes are implemented as B-trees, mapping sorted field values directly to document locations.

Q26. Does every MongoDB collection have at least one index, even with zero manual index creation? Explain.

SCORING ANSWER: Yes — every collection automatically has a default index on the `_id` field, created at collection creation time and never removable, which is why queries filtering by `_id` are always fast regardless of manual indexing.

Q27. In a compound index { branch: 1, cgpa: -1 }, why does it NOT efficiently support a query that filters ONLY on cgpa?

SCORING ANSWER: Because branch is the leading (first) field of the compound index, and the B-tree is primarily sorted by branch first. A query filtering only on cgpa can't use the index's sort order efficiently without also constraining branch — this is the basis of the ESR (Equality-Sort-Range) rule explored fully in Unit IV.

Q28. What is the write-performance trade-off of creating an index, and why shouldn't you index every field 'just in case'?

SCORING ANSWER: Every index must be updated on every insert/update/delete that touches its field(s), adding overhead to every write and consuming extra RAM/disk. Unnecessary indexes tax write performance permanently while providing no read benefit unless a query actually uses them — so indexes should be created deliberately, based on actual query patterns.

Q29. In a `.explain('executionStats')` output, what does it mean if `totalDocsExamined` is much larger than `nReturned`?

SCORING ANSWER: It signals an inefficient query — MongoDB had to examine far more documents than it actually needed to return, typically indicating a missing or unsuitable index for that query's filter, since an efficient IXSCAN would examine a number of documents close to the actual match count.

Q30. What does the `winningPlan.stage` value 'COLLSCAN' versus 'IXSCAN' tell you about a query's execution?

SCORING ANSWER: COLLSCAN means MongoDB performed a full collection scan, checking every document because no usable index existed for the query's filter. IXSCAN means MongoDB used an index to locate matching documents directly, which is almost always significantly faster on large collections.

STUDY TIP: Exam/viva technique: For every command-based question ('what does X do'), state the EFFECT first in one line, then the SYNTAX or a distinguishing detail second. Examiners specifically probe update-without-operator mistakes, skip()/pagination cost, and COLLSCAN-vs-IXSCAN reasoning — make sure these three are rock solid before your viva.

14. Frequently Confused Concepts — Quick Clarifications

The specific pairs of Unit II terms most often mixed up on exams and in interviews.

updateOne() (with \$set) vs. replaceOne()

updateOne() with an update operator modifies ONLY the specified fields, preserving everything else. replaceOne() swaps out the ENTIRE document body (except `_id`) — any field not explicitly included in the replacement is lost.

find() vs. findOne()

find() always returns a CURSOR (even for zero or one match), which must be iterated to see results. findOne() returns a single document directly (or null) — no cursor iteration needed.

\$set vs. no operator at all in an update call

\$set modifies only the given field(s), leaving the rest of the document untouched. Passing a plain field-value object with NO operator to updateOne()/updateMany() is actually invalid/behaves as a full replacement in some drivers — always wrap partial updates in \$set to avoid accidentally wiping a document.

deleteMany({}) vs. drop()

deleteMany({}) removes ALL documents but keeps the (now-empty) collection and its indexes intact. drop() removes the collection itself entirely, including all of its indexes — you'd need to recreate them afterward.

Implicit collection creation vs. db.createCollection()

Implicit creation happens automatically on first insert, but does NOT allow attaching options like schema validation at creation time. db.createCollection() is required whenever you need validators, capped-collection settings, or collation configured up front.

\$push vs. \$addToSet

\$push always appends, allowing duplicate values in the array. \$addToSet appends only if the value isn't already present, preventing duplicates — but note \$addToSet does NOT reorder or deduplicate existing duplicates already in the array.

Projection inclusion (field: 1) vs. query filter equality (field: value)

A projection's { field: 1 } controls which fields are RETURNED in matching documents. A query filter's { field: value } controls which documents MATCH in the first place. They serve entirely different purposes despite both appearing as arguments to find().

Index creation improving reads vs. its effect on writes

An index unambiguously speeds up READS that can use it (via IXSCAN instead of COLLSCAN). The same index unavoidably SLOWS DOWN writes slightly, because every insert/update/delete touching the indexed field(s) must also update the index's B-tree — indexes are a read/write trade-off, not a free performance upgrade.

Ordered vs. unordered insertMany()

Ordered (default: true) stops at the first failing document, leaving subsequent documents in the batch uninserted. Unordered (ordered: false) attempts every document regardless of earlier failures, reporting all outcomes together — favoring throughput over sequence guarantees.

COLLSCAN vs. IXSCAN in an explain() plan

COLLSCAN means every document in the collection was examined one by one (no usable index). IXSCAN means an index's B-tree was used to jump directly to matching entries — COLLSCAN is the red flag examiners and production engineers look for when diagnosing a slow query.

15. Full End-to-End Practical Walkthroughs

This section runs THREE complete, independent scenarios, each starting from the very first terminal command (installing/starting MongoDB, opening mongosh) all the way through database creation, full CRUD, and indexing with a verified explain() plan — combining everything from Unit I's environment setup through every concept in this unit. Type each command yourself in order; every command is copy-pasteable and every step is explained before you run it.

15.1 Walkthrough 1 — Retail Inventory System

Scenario: You're building the backend for a small retail store's inventory system, tracking products, stock counts, and prices.

Step 0 — Open a Terminal and Start MongoDB

First, make sure the MongoDB server (mongod) is actually running before trying to connect. This step assumes the Linux/macOS installation from Unit I, Section 6.

```
# On Linux (systemd-managed installation):
sudo systemctl start mongod
sudo systemctl status mongod          # confirm it shows 'active (running)'

# On macOS (Homebrew):
brew services start mongodb-community@7.0

# OR, run it manually in the foreground (any OS, useful for local dev):
mongod --dbpath /data/db
```

Why this step matters: mongosh is only a CLIENT — it has nothing to connect to until a mongod server process is actually listening on a port (default 27017). Skipping this step is the #1 reason beginners see a 'connection refused' error.

Step 1 — Open mongosh and Connect

```
# In a NEW terminal window/tab (leave mongod running in the first one)
mongosh

# You should see a prompt like:
# Current Mongosh Log ID: ...
# Connecting to: mongodb://127.0.0.1:27017/...
# Using MongoDB: 7.0.x
# test>
```

NOTE: The default prompt connects to a database named 'test' — this is just mongosh's default starting point, not a database you're forced to use. We'll switch to our own database in the next step.

Step 2 — Create the Database and an Explicitly Validated Collection

```

use retail_inventory

db.createCollection("products", {
  validator: {
    $jsonSchema: {
      bsonType: "object",
      required: ["sku", "name", "price", "stock"],
      properties: {
        sku: { bsonType: "string", description: "unique product code, required" },
        name: { bsonType: "string" },
        price: { bsonType: ["double", "int"], minimum: 0 },
        stock: { bsonType: "int", minimum: 0 }
      }
    }
  },
  validationAction: "error"
})

```

Explanation: we chose EXPLICIT creation (rather than letting the first insert create it implicitly) specifically because we need the validator attached from the very first document onward — this guarantees no bad data ever enters the products collection, even accidentally during early testing.

Step 3 — Create: Insert Initial Product Data

```

db.products.insertMany([
  { sku: "SKU-001", name: "Wireless Mouse", price: 599.00, stock: 120, category: "Electronics" },
  { sku: "SKU-002", name: "Mechanical Keyboard", price: 2499.00, stock: 45, category: "Electronics" },
  { sku: "SKU-003", name: "Notebook A5", price: 49.00, stock: 300, category: "Stationery" },
  { sku: "SKU-004", name: "Desk Lamp", price: 899.00, stock: 0, category: "Furniture" }
])

```

Explanation: insertMany() with all four required fields (sku, name, price, stock) present on every document satisfies the validator we attached in Step 2 — try removing a required field from one document and re-running to see the validation error firsthand.

Step 4 — Read: Query the Inventory

```

// All products, name and price only, sorted cheapest first
db.products.find({}, { name: 1, price: 1, _id: 0 }).sort({ price: 1 })

// Out-of-stock products (stock equals zero) -- needs restocking
db.products.find({ stock: 0 })

// Electronics under 1000 rupees
db.products.find({ category: "Electronics", price: { $lt: 1000 } })

```

```
// Count how many distinct categories worth of stock we're carrying
db.products.countDocuments({ stock: { $gt: 0 } })
```

Step 5 — Update: Simulate a Sale and a Restock

```
// A customer buys 3 wireless mice -- atomically decrement stock
db.products.updateOne({ sku: "SKU-001" }, { $inc: { stock: -3 } })

// The desk lamp gets restocked -- set stock directly, and log the restock date
db.products.updateOne(
  { sku: "SKU-004" },
  { $set: { stock: 50, lastRestockedAt: new Date() } }
)

// Apply a 10% price increase across ALL Electronics (bulk update)
db.products.updateMany({ category: "Electronics" }, { $mul: { price: 1.1 } })
```

Step 6 — Index: Optimize the Most Common Lookup (by SKU)

```
// BEFORE indexing -- check the current execution plan for a SKU lookup
// (this is how the checkout counter looks up a product by scanning its barcode)
db.products.find({ sku: "SKU-002" }).explain("executionStats")
// winningPlan.stage: "COLLSCAN"   <- scans all 4 documents (fine now, terrible at
500,000 SKUs)

// Create a UNIQUE index -- SKUs must never collide across products
db.products.createIndex({ sku: 1 }, { unique: true })

// AFTER indexing -- re-run the SAME query
db.products.find({ sku: "SKU-002" }).explain("executionStats")
// winningPlan.stage: "IXSCAN"
// executionStats.totalDocsExamined: 1   <- jumps directly to the one matching
product
```

Step 7 — Delete: Discontinue a Product (Safely)

```
// Step 7a: preview what will be removed
db.products.find({ sku: "SKU-003" })

// Step 7b: soft-delete preferred for retail (keeps historical sales records valid)
db.products.updateOne({ sku: "SKU-003" }, { $set: { discontinued: true,
discontinuedAt: new Date() } })

// Step 7c: exclude discontinued items from normal storefront queries going forward
db.products.find({ discontinued: { $ne: true } })
```

NOTE: End-to-end recap of Walkthrough 1: mongod start → mongosh connect → use + createCollection with a validator → insertMany → targeted find() queries with projection/sort → \$inc/\$set/\$mul updates → a unique index

verified via `explain()` → a safe soft-delete. Every single command chain here is directly reusable as a template for any inventory-style system you build in the future.

15.2 Walkthrough 2 — Hospital Patient Records System

Scenario: A small clinic needs to track patients, their visit history (embedded array), and prescriptions. This walkthrough emphasizes embedded arrays and \$elemMatch, which Walkthrough 1 didn't cover.

Step 0 — Start MongoDB and Connect (Atlas Cloud Version This Time)

To reinforce Unit I's cloud path too, this walkthrough connects to a MongoDB Atlas cluster instead of a local server — the CRUD syntax is 100% identical either way, only the connection step differs.

```
# Connect mongosh directly to an Atlas cluster
mongosh "mongodb+srv://<cluster-name>.mongodb.net/" --apiVersion 1 --username
<db_username>
# (enter your password when prompted)
```

Step 1 — Create Database and Collection

```
use clinic_records

db.createCollection("patients", {
  validator: {
    $jsonSchema: {
      bsonType: "object",
      required: ["name", "age", "gender"],
      properties: {
        name: { bsonType: "string" },
        age: { bsonType: "int", minimum: 0, maximum: 130 },
        gender: { enum: ["M", "F", "Other"] }
      }
    }
  }
})
```

NOTE: The enum keyword inside \$jsonSchema restricts a field to an exact whitelist of allowed values — here, gender must be exactly "M", "F", or "Other," and any other value is rejected by the validator at write time.

Step 2 — Create: Insert Patients With Embedded Visit History

```
db.patients.insertMany([
  {
    name: "Rohan Verma", age: 34, gender: "M",
    visits: [
      { date: ISODate("2026-06-10"), diagnosis: "Flu", doctor: "Dr. Kapoor" }
    ]
  },
  {
    name: "Anita Desai", age: 58, gender: "F",
    visits: [
      { date: ISODate("2026-03-02"), diagnosis: "Hypertension", doctor: "Dr. Rao" },
      { date: ISODate("2026-07-19"), diagnosis: "Hypertension", doctor: "Dr. Rao" }
    ]
  }
])
```

```

    ]
  }
})

```

Step 3 — Read: Query Nested Visit Data

```

// All patients over 50
db.patients.find({ age: { $gt: 50 } })

// Patients who were EVER diagnosed with Hypertension (searches inside the array)
db.patients.find({ "visits.diagnosis": "Hypertension" })

// Patients who saw Dr. Rao FOR Hypertension specifically -- same visit, both
conditions
// ($elemMatch required so both fields must match the SAME array element)
db.patients.find({ visits: { $elemMatch: { diagnosis: "Hypertension", doctor: "Dr.
Rao" } } })

// Patients with more than 1 recorded visit (frequent visitors)
db.patients.find({ visits: { $size: 1 } }) // exactly 1 visit
db.patients.find({ "visits.1": { $exists: true } }) // 2 or more visits (index 1
exists)

```

Step 4 — Update: Add a New Visit (Array Push) and Correct a Record

```

// A new visit is recorded for Rohan -- push into the visits array
db.patients.updateOne(
  { name: "Rohan Verma" },
  { $push: { visits: {
    date: new Date(), diagnosis: "Follow-up checkup", doctor: "Dr. Kapoor"
  } } }
)

// Correction: fix a typo'd doctor name across ALL of Anita's visit records
// (positional operator $[] applies to every matching array element)
db.patients.updateOne(
  { name: "Anita Desai" },
  { $set: { "visits.$[].doctor": "Dr. S. Rao" } }
)

```

NOTE: `$[]` is the ALL-POSITIONAL operator — it applies the update to EVERY element of the array, unlike the plain positional operator `$` (which updates only the FIRST array element matching your query filter). This distinction between `$` and `$[]` is a common point of confusion worth remembering.

Step 5 — Index: Speed Up Patient Lookups by Name

```

// Check current plan
db.patients.find({ name: "Anita Desai" }).explain("executionStats")
// winningPlan.stage: "COLLSCAN"

```

```
// Create the index
db.patients.createIndex({ name: 1 })

// Confirm the improvement
db.patients.find({ name: "Anita Desai" }).explain("executionStats")
// winningPlan.stage: "IXSCAN"
```

Step 6 — Delete: Remove a Single Erroneous Visit Entry

```
// Preview first
// (imagine a duplicate visit was accidentally recorded on 2026-07-19)
db.patients.find({ name: "Anita Desai" })

// $pull removes array elements matching a condition -- here, a specific date
db.patients.updateOne(
  { name: "Anita Desai" },
  { $pull: { visits: { date: ISODate("2026-07-19") } } }
)
```

NOTE: Notice deletion happened via an UPDATE (\$pull), not a top-level deleteOne() — because we wanted to remove one ELEMENT from an array field, not the whole patient document. Recognizing 'am I deleting a document or an array element within a document' is a key practical distinction this walkthrough is designed to drill in.

15.3 Walkthrough 3 — Employee HR Management System

Scenario: An HR system tracking employees, departments, and salaries — this walkthrough emphasizes compound indexing, upserts, and a full write-performance comparison (before/after indexing) using a larger generated dataset, closest to a real production diagnostic exercise.

Step 0 — Verify Your Connection (Any Method From Walkthroughs 1 or 2)

```
mongosh
db.runCommand({ ping: 1 }) // { ok: 1 } confirms the server is reachable
```

Step 1 — Create Database and Collection

```
use company_hr
db.createCollection("employees")
// (no validator this time -- deliberately, to show a schema-flexible collection in practice)
```

Step 2 — Create: Generate a Larger Sample Dataset Programmatically

Rather than typing hundreds of documents by hand, mongosh lets you use plain JavaScript to generate bulk sample data — extremely useful for realistically testing indexing performance, since a 5-document collection can't meaningfully demonstrate a COLLSCAN-vs-IXSCAN speed difference.

```
const departments = ["Engineering", "Sales", "HR", "Finance", "Marketing"]
let employeeBatch = []
for (let i = 1; i <= 2000; i++) {
  employeeBatch.push({
    employeeId: "EMP" + i,
    name: "Employee " + i,
    department: departments[i % departments.length],
    salary: 30000 + (i % 50) * 1000,
    active: true
  })
}
db.employees.insertMany(employeeBatch)
db.employees.countDocuments() // -> 2000
```

Step 3 — Read: Compare a Query BEFORE Any Index Exists

```
db.employees.find({ department: "Engineering", salary: { $gt: 50000 } })
  .explain("executionStats")
// winningPlan.stage: "COLLSCAN"
// executionStats.totalDocsExamined: 2000 <- examined EVERY document
// executionStats.executionTimeMillis: (small here, but scales badly at millions of docs)
```

Step 4 — Update: Give a Department-Wide Raise and Fix One Record (Upsert)

```
// 5% raise for the whole Engineering department
db.employees.updateMany({ department: "Engineering" }, { $mul: { salary: 1.05 } })

// Upsert: ensure an employee record exists for a newly onboarded hire,
// without needing to check first whether their record already exists
db.employees.updateOne(
  { employeeId: "EMP2001" },
  { $set: { name: "New Hire", department: "Engineering", salary: 45000, active: true } },
  { upsert: true }
)
```

Step 5 — Index: Create a Compound Index Matching the Query Pattern

Our Step 3 query filters on department (equality) AND salary (range). Following the equality-before-range ordering principle from Section 9.3, we put department first.

```
db.employees.createIndex({ department: 1, salary: 1 })
```

Step 6 — Read: Re-Run the SAME Query and Compare

```
db.employees.find({ department: "Engineering", salary: { $gt: 50000 } })
  .explain("executionStats")
// winningPlan.stage: "IXSCAN"
// executionStats.totalDocsExamined: close to nReturned (only Engineering employees
// checked)
// executionStats.executionTimeMillis: measurably lower than Step 3

// Bonus: verify the index exists and inspect all indexes on the collection
db.employees.getIndexes()
```

NOTE: This before/after pattern — run `explain()`, note `COLLSCAN` and a high `totalDocsExamined`, create the matching compound index, re-run the identical query, and observe `IXSCAN` with `totalDocsExamined` collapsing toward `nReturned` — is THE standard diagnostic workflow used by real backend/database engineers in production, and is very likely to appear directly in a practical exam or interview task.

Step 7 — Delete: Offboard an Employee (Safely)

```
// Preview
db.employees.find({ employeeId: "EMP2001" })

// Soft-delete (recommended for HR/payroll history)
db.employees.updateOne({ employeeId: "EMP2001" }, { $set: { active: false,
offboardedAt: new Date() } })

// Confirm active employee count going forward
db.employees.countDocuments({ active: true })
```

End of Unit II. When you're ready, say "start Unit 3" to continue into Schema Design and Data Modeling.